Day 35 Practice Problems: Git & GitHub Engineering Suite

This practice suite contains three production-grade engineering challenges designed to evaluate and solidify your mastery of branching architectures, merge conflict resolution, pull request lifecycles, and disaster recovery taught in Day 35.

Challenge 1: Production Hotfix & Branch Isolation Pipeline

Background & Context

You are an AI engineer working at an automated fintech startup. You are midway through training a new fraud detection model on a local feature branch named `feature-fraud-v2`. Suddenly, the Lead DevOps Engineer alerts you that an authentication vulnerability in production `main` must be patched immediately.

Requirements & Tasks

1. **Inspecting Active Status**:
    - Run the command to inspect your current branch and list all modified/untracked files.
2. **Safe Context Switching**:
    - Explain why running `git checkout main` with dirty, uncommitted changes might fail or accidentally bring uncommitted changes into `main`.
    - Stage and commit your current work-in-progress on `feature-fraud-v2` with an informative message.
3. **Hotfix Branch Creation & Deployment**:
    - Switch to `main` and ensure your local `main` matches the remote repository using the appropriate synchronization command.
    - Branch off `main` into a dedicated hotfix branch named `hotfix-auth-patch`.
    - Make the fix in `auth.py`, stage the file, and commit with message `"Patch CVE auth token bypass"`.
4. **Integration & Cleanup**:
    - Switch back to `main` and merge `hotfix-auth-patch`.
    - Push the updated `main` to the remote repository.
    - Delete the local hotfix branch `hotfix-auth-patch`.
    - Return to your original `feature-fraud-v2` branch to resume development.

Challenge 2: Multi-Developer Merge Conflict Resolution

Background & Context

You and a colleague (Engineer B) are working on the database configuration file `database.py`.

- Engineer B pushed a commit to `origin/main` updating the database port to `5432` and connection pool to `20`.
- Simultaneously, on your local machine's `main` branch, you committed changes to the exact same lines, setting the port to `5433` and pool size to `50`.

When you run `git pull origin main`, Git stops with:

```
CONFLICT (content): Merge conflict in database.py
Automatic merge failed; fix conflicts and then commit the result.
```

Requirements & Tasks

1. **Conflict Anatomy**:
    - Write out the exact anatomy of the conflict markers that Git inserts into `database.py`.
    - Clearly identify which section represents your local commit (`HEAD`) and which represents Engineer B's incoming commit.
2. **Resolution Protocol**:
    - Assume team policy dictates that the production database port must be `5432` (Engineer B's change) but the connection pool must be expanded to `50` (your change).
    - Show the final clean file content without any Git markers.
3. **Finalizing the Merge**:
    - Provide the complete sequence of commands to verify status, stage the resolved file, complete the merge commit, and push the resolution to GitHub.

Challenge 3: Disaster Recovery & Accidental Secret Commit Mitigation

Background & Context

A newly hired developer accidentally included an active OpenAI API secret key and a private database password inside a file named `.env`. They executed:

```
git add .
git commit -m "Add API endpoints and keys"
git push origin main
```

The repository is public on GitHub, exposing the secrets to automated web crawlers.

Requirements & Tasks

1. **Immediate Threat Neutralization**:
   - What is the very first and most critical action the team must take regarding the leaked credentials (outside of Git)?
2. **Untracking without Deleting Local File**:
   - If the developer simply runs `rm .env`, the file is deleted from their local computer, which breaks their local development environment.
   - What Git command untracks `.env` from Git's index while preserving the actual physical file on disk?
3. **Preventing Future Tracking**:
   - Write the exact entry required in `.gitignore` to ensure `.env` and all related environment configuration files (`.env.local`, `.env.production`) are never tracked again.
4. **Revert vs Reset Analysis**:
   - Compare `git revert <commit_hash>` vs `git reset --hard HEAD~1` in this public repository context. Explain why `git reset --hard` requires a dangerous force push (`git push --force`) and why secret credentials must always be revoked regardless of Git history rewriting.

Solutions Reference

For complete verified terminal commands and in-depth engineering explanations, review Solutions.md.